CUROVA Healthcare Web Application

Complete Backend Engineering Guide

Professional Reference for Viva Presentation

Backend Engineering Team
Capstone Project

March 28, 2026

Contents

Executive Summary **3**

1 Architecture & Philosophy **4**
1.1 Design Principles . . . 4
1.1.1 Separation of Concerns . . . 4
1.1.2 REST API Guidelines . . . 4
1.1.3 Token-Based Authentication . . . 5
1.1.4 Role-Based Access Control . . . 5
1.2 Data Flow Example: Patient Books Appointment . . . 6

2 Technology Stack Breakdown **7**
2.1 Backend Framework: Django . . . 7
2.1.1 Why Django? . . . 7
2.1.2 Django REST Framework (DRF) . . . 7
2.2 Database: PostgreSQL . . . 8
2.2.1 Why PostgreSQL? . . . 8
2.3 Web Server: Gunicorn . . . 8

3 Database Schema **9**
3.1 Core Entity Relationships . . . 9
3.1.1 User Hierarchy . . . 9
3.1.2 Appointment System . . . 9
3.1.3 Medical Records & Prescriptions . . . 10
3.2 Other Critical Models . . . 11
3.2.1 Notifications . . . 11
3.2.2 Lab Tests . . . 11

4 Authentication System **12**
4.1 Login Flow (Step-by-Step) . . . 12
4.1.1 Step 1: User Submits Credentials . . . 12
4.1.2 Step 2: Backend Validates . . . 12
4.1.3 Step 3: Token Generation . . . 13
4.1.4 Step 4: Client Storage . . . 13
4.1.5 Step 5: Authenticated Requests . . . 13
4.2 Token Lifecycle . . . 13
4.3 Permission Classes . . . 14

5 API Endpoints Reference **15**
5.1 Authentication Endpoints . . . 15
5.2 Appointment Endpoints . . . 15
5.2.1 Example: Getting Booked Slots . . . 15
5.3 Medical Records Endpoints . . . 16

5.4 Notifications Endpoints . . . 16

6 Key Design Patterns **17**
6.1 Serializers: Data Transformation . . . 17
6.2 Permission Classes: Authorization . . . 17
6.3 Pagination: Large Datasets . . . 18
6.4 Immutable Records: Audit Trails . . . 18
6.5 Unique Constraints: Preventing Duplicates . . . 19

7 Design Decisions & Rationale **20**
7.1 Why Django + Django REST Framework? . . . 20
7.1.1 The Alternatives . . . 20
7.1.2 Our Choice: Django + DRF . . . 20
7.2 Why Token-Based Authentication? . . . 21
7.2.1 Sessions vs. Tokens . . . 21
7.3 Why Role-Based Access Control? . . . 21
7.3.1 The Alternatives . . . 21
7.3.2 Why RBAC . . . 22
7.4 Why PostgreSQL? . . . 22
7.4.1 Alternatives . . . 22
7.4.2 Why PostgreSQL . . . 22
7.5 Why Immutable Prescriptions? . . . 23
7.5.1 The Alternative . . . 23
7.5.2 Why Immutable Wins . . . 23

8 Deployment Architecture **25**
8.1 Production Environment: Render + Vercel . . . 25
8.2 Deployment Pipeline . . . 25
8.3 Environment Variables . . . 26
8.4 Migrations on Production . . . 26
8.4.1 Automatic Migration (wsgi.py) . . . 26
8.5 Logs and Monitoring . . . 26
8.5.1 Render Backend Logs . . . 26

9 Testing & Debugging **28**
9.1 Common Error Messages . . . 28
9.1.1 400 Bad Request . . . 28
9.1.2 401 Unauthorized . . . 28
9.1.3 403 Forbidden . . . 29
9.1.4 404 Not Found . . . 29
9.1.5 500 Internal Server Error . . . 30
9.2 Django Shell Debugging . . . 30
9.3 CORS Debugging . . . 30

10 Code Walkthroughs **32**
10.1 Adding a New Endpoint: Patient Refill Request . . . 32
10.1.1 1. Create View . . . 32
10.1.2 2. Add URL Route . . . 33
10.1.3 3. Test the Endpoint . . . 33
10.2 Serializer with Validation . . . 33

Summary & Quick Reference **36**

Appendix: Interview Preparation **38**

Executive Summary

CUROVA is a production healthcare management web application built using Django REST Framework (backend) deployed on Render and React + Vite (frontend) deployed on Vercel. This document provides complete technical documentation covering architecture, design decisions, implementation details, and deployment considerations.

Key Facts:

- **Backend:** Django 6.0.2 + Django REST Framework 3.14.0
- **Database:** PostgreSQL (Render-managed)
- **Authentication:** Token-based (DRF TokenAuthentication)
- **Architecture:** RESTful API with role-based access control
- **Status:** Production deployment complete, all migrations applied
- **Users:** 4 roles (Patient, Doctor, Admin, Lab Technician)
- **Endpoints:** 30+ REST endpoints across 8 Django apps
- **Database:** 40+ migrations, normalized schema with ACID guarantees

Chapter 1

Architecture & Philosophy

1.1 Design Principles

1.1.1 Separation of Concerns

Each Django application handles a single domain of responsibility:

App	Domain	Responsibility
users	Authentication	Login, registration, user profiles, permissions
appointments	Scheduling	Appointment booking, slot management
medical	Medical Records	Doctor notes, patient history, prescriptions
medications	Medication Tracking	Patient medication management
documents	File Management	Medical document uploads
lab_tests	Lab Testing	Test orders, result reporting
notifications	In-app Alerts	Event notifications to users
messaging	(Placeholder)	For future messaging implementation

Table 1.1: Django Application Domains

Benefit: Changes to appointment logic don't affect notification system. Easier to test, maintain, and extend independently.

1.1.2 REST API Guidelines

All functionality exposed through HTTP endpoints following REST conventions:

```
GET    /api/appointments/              List all appointments
POST   /api/appointments/              Create appointment
GET    /api/appointments/5/            Get appointment #5
PATCH  /api/appointments/5/            Update appointment #5
DELETE /api/appointments/5/            Delete appointment #5
```

HTTP Status Codes:

- `200 OK` - Request succeeded

- `201 CREATED` - Resource created
- `400 BAD REQUEST` - Invalid client data
- `401 UNAUTHORIZED` - Authentication required
- `403 FORBIDDEN` - Permission denied
- `404 NOT FOUND` - Resource doesn't exist
- `500 INTERNAL SERVER ERROR` - Server error

1.1.3 Token-Based Authentication

Unlike traditional session-based authentication (cookies), CUROVA uses stateless token authentication:

1. User logs in with email and password
2. Server generates unique token (stored in database)
3. Client stores token in localStorage
4. Client sends token with every request via Authorization header
5. Server validates token and identifies user

Advantages:

- Stateless - no server session memory needed
- Works across domains (perfect for SPA + separate API)
- Scales horizontally - any backend server can validate token
- Frontend agnostic - could be web, mobile, desktop

1.1.4 Role-Based Access Control

Users assigned one of four roles controlling permissions:

```
class User(models.Model):
    user_type = CharField(choices=[
        "patient",     # Book appointments, view own records
        "doctor",      # Manage schedule, create records
        "admin",       # System oversight
        "lab_tech"     # Enter lab results
    ])
```

Permission classes enforce role restrictions:

```
@permission_classes([IsAuthenticated, IsDoctor])
def doctor_only_view(request):
    # Only doctors can access
    pass
```

1.2 Data Flow Example: Patient Books Appointment

1. **Frontend:** User selects doctor, date, and time
2. **Frontend:** Fetches booked slots for that date via
   `GET /api/appointments/booked-slots/?doctor_id=5&date=2026-04-15`
3. **Frontend:** User clicks available time, submits form
4. **Backend:** Validates in this order:
   (a) Is user authenticated? (check token)
   (b) Is user a patient? (check user_type)
   (c) Is doctor available on that day? (check available_days)
   (d) Is time within working hours? (check working_hours_start/end)
   (e) Is time on slot boundary? (check slot_duration alignment)
   (f) Is slot not already booked? (check unique constraint)
5. **Backend:** Creates Appointment record in database
6. **Backend:** Creates notifications for both doctor and patient
7. **Backend:** Returns 201 CREATED with appointment details
8. **Frontend:** Shows success message, updates appointment list

Chapter 2

Technology Stack Breakdown

2.1 Backend Framework: Django

Django is a Python web framework providing:

- **ORM (Object-Relational Mapping)** - Write Python, not SQL
- **Admin Panel** - Auto-generated CRUD interface
- **Security** - CSRF protection, SQL injection prevention
- **Migrations** - Database schema versioning via git
- **Authentication** - Built-in user model and auth system

2.1.1 Why Django?

ORM Benefits:

```python
# Safe from SQL injection
user = User.objects.get(email="john@curova.com")

# Django generates parameterized SQL automatically
# No string concatenation = no injection vulnerability
```

Admin Panel:

```
Default: http://localhost:8000/admin/
- View and edit all database records
- User management, permissions
- No additional code needed
```

2.1.2 Django REST Framework (DRF)

DRF extends Django with REST capabilities:

- `@api_view` decorator for HTTP methods
- Serializers for JSON conversion and validation
- Permission classes for authorization
- Throttling for rate limiting
- CORS support

2.2 Database: PostgreSQL

2.2.1 Why PostgreSQL?

ACID Compliance: Guarantees data integrity even on failures

- Atomicity - All-or-nothing transactions
- Consistency - Valid state always maintained
- Isolation - Concurrent requests don't interfere
- Durability - Data survives crashes

Advanced Features:

- JSON fields with indexing and validation
- Full-text search capabilities
- Array types for complex data
- Point-in-time recovery for backups

Production Ready:

- Replication for high availability
- Monitoring and logging
- Encryption at rest and in transit
- HIPAA compliance compatible

2.3 Web Server: Gunicorn

Gunicorn is the production WSGI application server:

```
Development: python manage.py runserver
    - Single-threaded
    - Fast for development
    - Handles 1 user at a time

Production: gunicorn curova_backend.wsgi:application
    - Multi-process (handles 100+ concurrent requests)
    - Production-ready
    - Can restart on crash
    - Monitorable via service
```

Chapter 3

Database Schema

3.1 Core Entity Relationships

3.1.1 User Hierarchy

All users (patient, doctor, admin) inherit from Django's AbstractUser:

```
class User(AbstractUser):
    email = EmailField(unique=True)  # Login username
    username = CharField()            # For @mentions
    password = CharField()            # PBKDF2 hashed
    first_name, last_name = CharField()
    user_type = CharField(choices=['patient', 'doctor', 'admin', 'lab_tech'])
    is_active = BooleanField()       # Can be deactivated
    created_at = DateTimeField(auto_now_add=True)

class Patient(models.Model):
    user = OneToOneField(User)       # 1:1 relationship
    DOB = DateField()
    gender = CharField()
    blood_type = CharField()
    allergies = JSONField()          # Array of strings
    chronic_conditions = JSONField() # Array of conditions
    medical_history = JSONField()    # Complex object

class Doctor(models.Model):
    user = OneToOneField(User)
    license_number = CharField(unique=True)
    specialization = CharField(choices=[...14 options...])
    years_experience = IntegerField()
    consultation_fee = DecimalField()
    available_days = JSONField()     # ["Monday", "Tuesday", ...]
    working_hours_start = TimeField()
    working_hours_end = TimeField()
    slot_duration = IntegerField()   # Minutes per appointment slot
```

3.1.2 Appointment System

```python
class Appointment(models.Model):
    STATUS_CHOICES = [
        'scheduled',     # Booked but not confirmed
        'confirmed',     # Confirmed by both parties
        'in_progress',   # Currently happening
        'completed',     # Finished
        'cancelled',     # Cancelled by either party
        'no_show',       # Didn't happen
        'rescheduled'    # Moved to different time
    ]

    patient = ForeignKey(Patient, on_delete=models.CASCADE)
    doctor = ForeignKey(Doctor, on_delete=models.CASCADE)
    appointment_date = DateField()
    appointment_time = TimeField()
    status = CharField(choices=STATUS_CHOICES)
    reason = TextField()

    class Meta:
        # Prevents double-booking (database enforces)
        unique_together = ('doctor', 'appointment_date', 'appointment_time')
```

Key Feature: UNIQUE constraint at database level prevents race conditions. Even if multiple servers try to book same slot simultaneously, only one succeeds.

3.1.3 Medical Records & Prescriptions

```python
class MedicalRecord(models.Model):
    patient = ForeignKey(Patient)
    doctor = ForeignKey(Doctor)
    appointment = ForeignKey(Appointment, null=True, blank=True)

    chief_complaint = TextField()
    diagnosis = JSONField()            # Array of diagnoses
    symptoms = JSONField()             # Array of symptoms
    examination_notes = TextField()
    treatment_plan = TextField()
    vitals = JSONField()               # {BP: "120/80", HR: 72, Temp: 98.6}
    follow_up_date = DateField(null=True)

class Prescription(models.Model):
    # IMMUTABLE - no edits allowed after creation
    medical_record = ForeignKey(MedicalRecord)
    medication_name = CharField()
    dosage = CharField()               # "500mg"
    frequency = CharField()            # "Twice daily"
    duration = CharField()             # "7 days"
    instructions = TextField()
    created_at = DateTimeField(auto_now_add=True)  # Never changes
```

Why Separated?

- Prescription = Legal document, immutable, audit trail
- Medication = What patient is actively taking, mutable
- One prescription might create multiple medication entries

3.2 Other Critical Models

3.2.1 Notifications

```
class Notification(models.Model):
    TYPE_CHOICES = [
        'appointment',     # Booking/confirmation
        'medication',      # Reminder
        'lab_result',      # Results ready
        'prescription',    # New prescription
        'system'           # General alerts
    ]

    recipient = ForeignKey(User)
    title = CharField()                # Short title
    message = TextField()              # Full message
    notification_type = CharField(choices=TYPE_CHOICES)
    is_read = BooleanField(default=False)
    related_object_type = CharField() # "Appointment", "LabTest", etc
    related_object_id = IntegerField()
    created_at = DateTimeField(auto_now_add=True)
```

Usage: Single source of truth for all notifications. Appointment changes, lab results, medications all create Notification entries.

3.2.2 Lab Tests

```
class LabTest(models.Model):
    STATUS_CHOICES = ['ordered', 'completed']

    patient = ForeignKey(Patient)
    doctor = ForeignKey(Doctor)        # Who ordered it
    test_type = CharField()            # "CBC", "Lipid Panel", etc
    status = CharField(choices=STATUS_CHOICES)
    created_at = DateTimeField(auto_now_add=True)

class LabTestResult(models.Model):
    lab_test = OneToOneField(LabTest)
    results = JSONField()              # Measured values
    findings = TextField()             # Abnormalities noted
    impression = TextField()           # Clinical interpretation
    uploaded_by = ForeignKey(User)     # Lab technician
    uploaded_at = DateTimeField(auto_now_add=True)
```

Chapter 4

Authentication System

4.1 Login Flow (Step-by-Step)

4.1.1 Step 1: User Submits Credentials

Frontend sends:

```
POST /api/auth/login/
{
    "email": "doctor@curova.com",
    "password": "securepass123"
}
```

4.1.2 Step 2: Backend Validates

```
@api_view(["POST"])
@permission_classes([AllowAny])
def login_view(request):
    serializer = LoginSerializer(data=request.data)
    serializer.is_valid(raise_exception=True)  # Validates
   format & creds

    user = serializer.validated_data["user"]
    token, _ = Token.objects.get_or_create(user=user)

    return Response({
        "token": token.key,
        "user": UserSerializer(user).data,
    })
```

Serializer validates:

1. Email format is valid
2. User exists in database
3. Password matches (hashed comparison)
4. User is active (not deactivated)

4.1.3 Step 3: Token Generation

```
token = Token.objects.create(user=user)
# Generates 40-character random string
# Example: "a3b4c5d6e7f8g9h0i1j2k3l4m5n6o7p8q9r0s1t2"
```

Stored in database:

token_key	user_id	created_at
a3b4c5d6...	5	2026-03-28 14:30:00

Table 4.1: Token Storage

4.1.4 Step 4: Client Storage

Frontend stores token:

```
localStorage.setItem("token", "a3b4c5d6...");
```

4.1.5 Step 5: Authenticated Requests

Every API call includes token:

```
const token = localStorage.getItem("token");
const headers = {
    "Authorization": `Token ${token}`
};

axios.get("/api/appointments/", {headers});
```

HTTP Header sent:

```
Authorization: Token a3b4c5d6e7f8g9h0i1j2k3l4m5n6o7p8q9r0s1t2
```

4.2 Token Lifecycle

```
1. User logs in
   ↓
2. Token created: {key: "abc123...", user_id: 5, created: 2026-03-28}
   ↓
3. Token stored in localStorage
   ↓
4. On every API call, token sent in Authorization header
   ↓
5. Server validates: "Is this token in database? Is user active?"
   ↓
6. Yes → request.user = User(id=5), grant access
   No → Return 401 Unauthorized → Frontend redirects to login
   ↓
7. User logs out
```

```
   ↓
8. Token.objects.delete(user=user_id) removes token from database
   ↓
9. Next API call with old token → Server can't find it → 401
```

4.3 Permission Classes

```python
# Example: IsPatient permission class
class IsPatient(BasePermission):
    """Allow access only to authenticated patients."""

    def has_permission(self, request, view):
        return (
            request.user and
            request.user.is_authenticated and
            request.user.user_type == "patient"
        )

# Usage:
@api_view(['GET'])
@permission_classes([IsAuthenticated, IsPatient])
def patient_dashboard(request):
    # If not authenticated     401
    # If authenticated but not patient     403 Forbidden
    # If patient     200 OK, proceed
    pass
```

Two-level authorization:

1. **View-level:** IsAuthenticated checks if logged in
2. **View-level:** IsPatient checks if correct role
3. **Object-level:** (optional) Check if owns object

Chapter 5

API Endpoints Reference

5.1 Authentication Endpoints

Method	Endpoint	Auth	Purpose
POST	/api/auth/register/	None	Create account
POST	/api/auth/login/	None	Get token
POST	/api/auth/logout/	Required	Invalidate token
GET	/api/auth/me/	Required	Current user info
PUT	/api/auth/change-password/	Required	Change password
GET	/api/auth/doctors/	None	Public doctor list
GET	/api/auth/profile/	Patient	Get patient profile
PUT	/api/auth/profile/	Patient	Update profile
GET	/api/auth/doctor/profile/	Doctor	Get doctor profile

Table 5.1: Authentication Endpoints

5.2 Appointment Endpoints

Method	Endpoint	Auth	Purpose
GET	/api/appointments/	Required	List my appointments
POST	/api/appointments/	Patient	Book appointment
GET	/api/appointments/5/	Required	Get specific appointment
PATCH	/api/appointments/5/	Required	Update status
GET	/api/appointments/booked-slots/	None	See available times

Table 5.2: Appointment Endpoints

5.2.1 Example: Getting Booked Slots

```
GET /api/appointments/booked-slots/?doctor_id=5&date
    =2026-04-15

Response:
{
    "booked_times": [
        "09:00",
        "09:30",
        "14:00"
    ]
```

```
}
```

Frontend uses this to gray out already-booked times in the calendar.

5.3 Medical Records Endpoints

Method	Endpoint	Auth	Purpose
GET	/api/records/	Required	List my records
POST	/api/records/	Doctor	Create record
GET	/api/records/5/	Required	Get specific record

Table 5.3: Medical Records Endpoints

5.4 Notifications Endpoints

Method	Endpoint	Auth	Purpose
GET	/api/notifications/	Required	List my notifications
PATCH	/api/notifications/5/read/	Required	Mark as read
POST	/api/notifications/mark-all-read/	Required	Mark all read
GET	/api/notifications/unread-count/	Required	Count unread
DELETE	/api/notifications/5/	Required	Delete notification

Table 5.4: Notifications Endpoints

Chapter 6

Key Design Patterns

6.1 Serializers: Data Transformation

Problem: Database returns Python objects, API returns JSON

```python
# Database model
class Doctor(models.Model):
    user = OneToOneField(User)
    specialization = CharField()

# Serializer
class DoctorSerializer(serializers.ModelSerializer):
    user_name = serializers.CharField(source='user.first_name'
    )

    class Meta:
        model = Doctor
        fields = ['id', 'specialization', 'user_name']

# Usage
doctor = Doctor.objects.get(id=5)
serializer = DoctorSerializer(doctor)
serializer.data  # Python dict
json.dumps(serializer.data)  # JSON string for HTTP response
```

6.2 Permission Classes: Authorization

Pattern: Reusable, composable authorization checks

```python
# Instead of repeating this in every view:
if request.user.user_type != "doctor":
    return Response({"error": "Forbidden"}, status=403)

# Create reusable class:
class IsDoctor(BasePermission):
    def has_permission(self, request, view):
        return request.user.user_type == "doctor"

# Use anywhere:
@permission_classes([IsAuthenticated, IsDoctor])
def doctor_only_view(request):
```

```
    pass
```

6.3 Pagination: Large Datasets

Problem: 10,000 notifications at once = slow network + memory issues

Solution: Return 20 at a time with navigation

```
GET /api/notifications/?page=1

{
    "count": 10000,
    "next": "http://.../api/notifications/?page=2",
    "previous": null,
    "results": [...20 items...]
}

GET /api/notifications/?page=2

{
    "count": 10000,
    "next": "http://.../api/notifications/?page=3",
    "previous": "http://.../api/notifications/?page=1",
    "results": [...20 items...]
}
```

Benefits:

- Network: 5MB → 100KB per request (50x smaller)
- Database: Loads 20 records instead of 10,000
- User Experience: Instant first page load

6.4 Immutable Records: Audit Trails

Design: Prescriptions cannot be edited after creation

```
class Prescription(models.Model):
    # No update endpoint allowed
    # No serializer with PUT/PATCH
    # Views only support GET and POST

    medication_name = CharField()
    dosage = CharField()
    created_at = DateTimeField(auto_now_add=True)
```

Rationale:

1. **Legal:** Proof of what was prescribed on which date
2. **Audit Trail:** Complete history for compliance
3. **Safety:** Forces handling of corrections properly

6.5 Unique Constraints: Preventing Duplicates

Design: Database prevents double-booking at lowest level

```
class Appointment(models.Model):
    doctor = ForeignKey(Doctor)
    appointment_date = DateField()
    appointment_time = TimeField()

    class Meta:
        unique_together = ('doctor', 'appointment_date', '
    appointment_time')
```

Why database-level?

1. **Race conditions:** Two requests simultaneously check availability
   (a) Request A: Check if 2:00 PM free → Yes
   (b) Request B: Check if 2:00 PM free → Yes (still, before A creates)
   (c) Request B: Create appointment → Success
   (d) Request A: Create appointment → DB says NO (duplicate)
   (e) Result: Only one created, other gets error
2. **Application bug safety:** If code has bug, DB catches it
3. **Multi-deployment:** Works with multiple servers sharing database

Chapter 7

Design Decisions & Rationale

7.1 Why Django + Django REST Framework?

7.1.1 The Alternatives

- Flask (lightweight Python)
- Node.js Express (JavaScript)
- FastAPI (modern Python)
- Spring Boot (Java)

7.1.2 Our Choice: Django + DRF

1. Built-in ORM prevents security issues

```
# Django ORM is safe
user = User.objects.get(email=user_input)  # Parameterized -
    safe

# Raw SQL would be dangerous
cursor.execute(f"SELECT * FROM users WHERE email = '{
    user_email}'")
# SQL injection if user_email = "' OR '1'='1"
```

2. Admin Panel saves weeks of development
Default admin interface at `/admin/`:

- View all database records
- Create test data
- User management
- Permission administration
- Zero extra code needed

3. Migrations enable safe deployments

1. Developer: Add field to model
2. Django: Generate migration file

3. Production: Run migration - schema updates automatically
4. Result: Zero-downtime deployments

4. DRF provides REST conventions

- Token authentication
- Permission classes
- Serializers for validation
- Throttling for rate limiting
- CORS support

All built-in, not requiring external libraries.

7.2 Why Token-Based Authentication?

7.2.1 Sessions vs. Tokens

Aspect	Sessions	Tokens
Storage	Server memory (Redis)	Database
Domains	Cookies (same domain only)	Headers (any domain)
Scaling	Needs sticky sessions	Stateless, scales horizontally
Decoupling	Tight coupling	Frontend + Backend independent

Table 7.1: Sessions vs. Tokens

Our Architecture:

- Frontend: https://curovafrontend.vercel.app
- Backend: https://curova-backend.onrender.com
- Different domains → Sessions won't work
- Tokens work perfectly across domains

7.3 Why Role-Based Access Control?

7.3.1 The Alternatives

- **No permissions:** Anyone can see anyone's data (insecure)
- **Attribute-based (ABAC):** Complex rules combining many attributes
- **Role-based (RBAC):** Simple 4 roles with clear permissions

7.3.2 Why RBAC

Healthcare needs are clear:

- **Patient:** Can book appointments, see own records
- **Doctor:** Can manage schedule, see patients, create records
- **Admin:** Can see all users and statistics
- **Lab Tech:** Can upload lab test results

RBAC Implementation:

```
# View-level: Restrict entire endpoint
@permission_classes([IsAuthenticated, IsDoctor])
def doctor_endpoint(request):
    pass  # Can't even enter unless doctor

# Object-level: Can see object only if own it
if not isPatientOwnDoctor(doctor, request.user):
    return 403 Forbidden
```

7.4 Why PostgreSQL?

7.4.1 Alternatives

- SQLite (file-based, single-user)
- MySQL (similar to PostgreSQL)
- MongoDB (NoSQL, document-based)
- Firebase (managed, serverless)

7.4.2 Why PostgreSQL

1. ACID Compliance (Data Integrity)
Example:

1. Check if appointment slot is free
2. Create appointment
3. Create notification

What if step 2 fails?

- Without ACID: Step 1 succeeded, step 2 failed, database in inconsistent state
- With ACID (PostgreSQL): Either all 3 succeed or all 3 fail
- No partial state possible

Critical for healthcare:

- Prescription can't exist without medical record

- Appointment can't be double-booked
- Patient can't see other patients' data (foreign key constraint)

2. Advanced Data Types
Our models use JSON fields:

```
class Patient(models.Model):
    allergies = JSONField()            # ["Penicillin", "Shellfish"]
    chronic_conditions = JSONField()   # ["Diabetes", "Hypertension"]
```

PostgreSQL provides:

- Native JSON support with indexing
- Query into JSON structure
- Validation of JSON format

SQLite cannot index JSON fields - queries would be slow.
3. Production-Ready Features

- Replication for backups
- Point-in-time recovery
- Encryption at rest and in transit
- User authentication
- Audit logging

All required for HIPAA compliance (healthcare data protection).

7.5 Why Immutable Prescriptions?

7.5.1 The Alternative

Allow doctors to edit prescriptions after creation.

7.5.2 Why Immutable Wins

1. Legal Compliance

```
Medical Board Audit:
"Show me what was prescribed on March 15"

Mutable: Could have been edited → unclear what was original
Immutable: Exact historical record → clear proof
```

2. Audit Trail

```
Patient has adverse reaction:
"What medication caused this?"

Mutable: DB says "Aspirin" but doctor edited from "Ibuprofen"
        Who's liable? Unclear
Immutable: Prescription explicitly shows "Ibuprofen" as original
          Clear liability assignment
```

3. Safety

```
Doctor prescribes Aspirin → Sent to pharmacy
Pharmacy fills Aspirin
Doctor realizes mistake, edits database to Ibuprofen
Patient gets Aspirin (already filled) but DB says Ibuprofen
Medication mismatch → Patient harmed
```

Correct approach (immutable):

```python
# Keep original (immutable)
original = Prescription.objects.get(id=123)

# Create corrected version
corrected = Prescription.objects.create(
    medical_record=original.medical_record,
    medication_name="Ibuprofen",  # Corrected
    notes="CORRECTION: Previous was Aspirin"
)
```

Both exist in database forever - clear history.

Chapter 8

Deployment Architecture

8.1 Production Environment: Render + Vercel

Component	Platform	URL
Backend (Django)	Render	https://curova-backend.onrender.com
Frontend (React)	Vercel	https://curovafrontend.vercel.app
Database (PostgreSQL)	Render-managed	Included with backend

Table 8.1: Production Deployment

8.2 Deployment Pipeline

1. Developer pushes code to GitHub

```
git push origin main

```

2. Render webhook receives notification

3. Render automatically:

   (a) Pulls latest code from GitHub

   (b) Installs dependencies: `pip install -r requirements.txt`

   (c) Runs migrations: `python manage.py migrate`

   (d) Starts server: `gunicorn curova_backend.wsgi:application`

4. Backend is live at https://curova-backend.onrender.com

5. Frontend (Vercel) similarly auto-deploys on code push

8.3 Environment Variables

Variable	Platform	Purpose
SECRET_KEY	Render	Django signing key
DEBUG	Render	Debug mode (False in production)
DATABASE_URL	Render	PostgreSQL connection string
GOOGLE_CLIENT_ID	Render	Google OAuth credential
CORS_ALLOWED_ORIGINS	Render	Frontend domain(s)
VITE_API_URL	Vercel	Backend API endpoint

Table 8.2: Environment Variables

Security: Never hardcode secrets. All sensitive values stored as environment variables on deployment platform.

8.4 Migrations on Production

8.4.1 Automatic Migration (wsgi.py)

```python
# /backend/curova_backend/wsgi.py
import os
import sys
from django.core.management import call_command
from django.core.wsgi import get_wsgi_application

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'curova_backend.settings')

# Auto-run migrations on startup
if 'runserver' not in sys.argv and 'manage.py' not in sys.argv:
    try:
        call_command('migrate', '--noinput', verbosity=1)
        print("Migrations applied successfully")
    except Exception as e:
        print(f"Migration error (continuing): {e}")

application = get_wsgi_application()
```

Why this approach:

- Zero-downtime deployments
- Gunicorn starts after migrations complete
- Automatic on every deployment
- No manual intervention needed

8.5 Logs and Monitoring

8.5.1 Render Backend Logs

`Dashboard → Service → Logs`

```
Shows:
- HTTP requests and responses
- Python errors and tracebacks
- Migration output
- Server startup messages
```

Recent successful deploy log:

```
[2026-03-28 16:58:41 +0000] [56] [INFO] Starting gunicorn 23.0.0

Operations to perform:
  Apply all migrations: admin, appointments, auth, authtoken, ...

Running migrations:
  No migrations to apply.   ← Migrations successful

127.0.0.1 - - [28/Mar/2026:16:58:43 +0000]
    "GET /api/auth/doctors/?page_size=6 HTTP/1.1" 200 2
    ← Status 200 = successful response
```

Chapter 9

Testing & Debugging

9.1 Common Error Messages

9.1.1 400 Bad Request

Cause: Invalid request data

```
{
  "email": ["This field may not be blank."],
  "password": ["Ensure this field has at least 8 characters."]
}
```

Debugging:

1. Check JSON syntax is valid
2. Verify all required fields present
3. Validate data types match

```
# Bad: Sending string instead of integer
{
  "doctor": "5"  # Should be number
}

# Good:
{
  "doctor": 5
}
```

9.1.2 401 Unauthorized

Cause: Missing or invalid authentication

```
{"detail": "Authentication credentials were not provided."}
```

Debugging:

1. Is Authorization header present?
2. Is token format correct? `Authorization: Token abc123...`
3. Has user deleted account or logged out?

4. Check token exists in database

```
python manage.py shell
>>> from rest_framework.authtoken.models import Token
>>> Token.objects.filter(key="abc123...")
<QuerySet [<Token: abc123...>]>
```

9.1.3 403 Forbidden

Cause: Authenticated but lacks permission

```
{"detail": "Permission denied."}
```

Debugging:

1. Check user role: Is it required role?

```
python manage.py shell
>>> from users.models import User
>>> user = User.objects.get(email="patient@curova.com")
>>> user.user_type
'patient'  # Not 'doctor' - that's why 403!
```

2. Check permission class requirements

```
@permission_classes([IsDoctor])  # endpoint requires
    doctor
# but user is patient     403

```

9.1.4 404 Not Found

Cause: Resource doesn't exist

Debugging:

1. Check resource exists in database

```
python manage.py shell
>>> from appointments.models import Appointment
>>> Appointment.objects.filter(id=999)  # Does it exist?
<QuerySet []>  # Nope - that's why 404
```

2. Verify correct endpoint URL

3. Check spelling of resource ID

9.1.5 500 Internal Server Error

Cause: Backend code crashed

Debugging:

1. Check Render logs for Python traceback

```
Traceback (most recent call last):
  File "django/core/handlers/wsgi.py", line 111, in __call__
    ...
  KeyError: 'appointment_date'
```

2. Common causes:
   - Missing database field
   - Typo in code
   - Null value where not expected
   - Database connection down

9.2 Django Shell Debugging

```
python manage.py shell

# Query data
>>> from users.models import User, Patient
>>> patients = Patient.objects.all()
<QuerySet [<Patient: John's profile>, ...]>

# Get specific
>>> user = User.objects.get(email="doctor@curova.com")
>>> user.user_type
'doctor'

# Count records
>>> Appointment.objects.count()
47

# Filter
>>> from appointments.models import Appointment
>>> Appointment.objects.filter(
...     status='completed',
...     appointment_date__gte='2026-03-01'
... ).count()
12
```

9.3 CORS Debugging

Error: `blocked by CORS policy`

```
# Test CORS headers with curl
curl -X OPTIONS https://curova-backend.onrender.com/api/
    appointments/ \
```

```
  -H "Origin: https://curovafrontend.vercel.app" \
  -H "Access-Control-Request-Method: POST" \
  -v

# Should show:
# Access-Control-Allow-Origin: https://curovafrontend.vercel.
    app
# Access-Control-Allow-Methods: GET, POST, PATCH, DELETE
```

If not showing, check CORS_ALLOWED_ORIGINS in settings.py.

Chapter 10

Code Walkthroughs

10.1 Adding a New Endpoint: Patient Refill Request

10.1.1 1. Create View

```python
# /backend/medical/views.py

@api_view(['POST'])
@permission_classes([IsAuthenticated, IsPatient])
def request_prescription_refill(request):
    """POST /api/records/prescriptions/{id}/refill-request/"""

    prescription_id = request.parser_context['kwargs']['pk']

    try:
        prescription = Prescription.objects.get(id=prescription_id)
    except Prescription.DoesNotExist:
        return Response(
            {'error': 'Prescription not found'},
            status=status.HTTP_404_NOT_FOUND
        )

    # Verify patient owns prescription
    patient = request.user.patient_profile
    if prescription.medical_record.patient != patient:
        return Response(
            {'error': 'Cannot request refill for someone elses prescription'},
            status=status.HTTP_403_FORBIDDEN
        )

    # Create refill request
    refill = PrescriptionRefillRequest.objects.create(
        prescription=prescription,
        patient=patient
    )

    # Notify doctor
    from notifications.helpers import create_notification
    doctor = prescription.medical_record.doctor
    create_notification(
        recipient=doctor.user,
        title=f'Prescription refill request',
```

```
        message=f'{patient.user.first_name} requested refill of {prescription.medication_name}',
        notification_type='prescription'
    )

    return Response(
        {'id': refill.id, 'status': 'pending'},
        status=status.HTTP_201_CREATED
    )
```

10.1.2 2. Add URL Route

```
# /backend/medical/urls.py

urlpatterns = [
    path('records/', views.medical_record_list),
    path('records/<int:pk>/', views.medical_record_detail),
    # Add this:
    path('records/prescriptions/<int:pk>/refill-request/',
         views.request_prescription_refill,
         name='prescription-refill-request'),
]
```

10.1.3 3. Test the Endpoint

```
# Get patient token
curl -X POST https://curova-backend.onrender.com/api/auth/login/ \
  -H "Content-Type: application/json" \
  -d '{"email":"patient@curova.com","password":"pass123"}'

# Response includes token
# "token": "abc123..."

# Request refill
curl -X POST https://curova-backend.onrender.com/api/records/prescriptions/5/refill-request/ \
  -H "Authorization: Token abc123..." \
  -H "Content-Type: application/json" \
  -d '{}'

# Response
# {"id": 1, "status": "pending", ...}
```

10.2 Serializer with Validation

```
# /backend/appointments/serializers.py

class CreateAppointmentSerializer(serializers.Serializer):
```

```
    doctor = serializers.PrimaryKeyRelatedField(
        queryset=Doctor.objects.all()
    )
    appointment_date = serializers.DateField()
    appointment_time = serializers.TimeField()
    reason = serializers.CharField(max_length=500)

    def validate_appointment_date(self, value):
        """Validate date field individually."""
        if value < timezone.now().date():
            raise serializers.ValidationError(
                "Cannot book in the past"
            )
        return value

    def validate(self, data):
        """Validate across multiple fields."""
        doctor = data['doctor']
        date = data['appointment_date']
        time = data['appointment_time']

        # Check 1: Doctor available that day
        day_name = date.strftime('%A').lower()
        if day_name not in doctor.available_days:
            raise serializers.ValidationError({
                'appointment_date': f"Doctor not available on {day_name}s"
            })

        # Check 2: Time within working hours
        if not (doctor.working_hours_start <= time <= doctor.working_hours_end):
            raise serializers.ValidationError({
                'appointment_time': "Outside working hours"
            })

        # Check 3: Time on slot boundary (e.g., 30-min slots)
        start_seconds = (
            doctor.working_hours_start.hour * 3600 +
            doctor.working_hours_start.minute * 60
        )
        time_seconds = time.hour * 3600 + time.minute * 60
        offset = (time_seconds - start_seconds) % (doctor.slot_duration * 60)

        if offset != 0:
            raise serializers.ValidationError({
                'appointment_time': f"Must align to {doctor.slot_duration}-min boundary"
            })

        # Check 4: Slot not already booked
        is_taken = Appointment.objects.filter(
            doctor=doctor,
            appointment_date=date,
            appointment_time=time,
            status__in=['scheduled', 'confirmed']
        ).exists()
```

```

        if is_taken:
            raise serializers.ValidationError({
                'appointment_time': "Slot already booked"
            })

        return data
```

Summary & Quick Reference

Key Takeaways

1. Architecture

- RESTful API with Django + DRF
- Token-based authentication
- Role-based access control (4 roles)
- Separated frontend and backend for independent scaling

2. Database

- PostgreSQL with ACID guarantees
- 40+ migrations for schema versioning
- Relationships carefully designed
- Immutable records for audit trail (prescriptions)

3. Security

- ORM prevents SQL injection
- Hashed passwords (PBKDF2)
- CORS properly configured
- Token validation on every request
- Permission classes for authorization

4. Production

- Deployed to Render (backend) + Vercel (frontend)
- Automatic deployments from GitHub
- Environment variables for secrets
- Auto-migrations on startup

File Structure Quick Reference

```
backend/
 curova_backend/
    settings.py       ← Database, apps, security config
    urls.py           ← Root URL routing
    wsgi.py           ← Auto-migration on startup

 users/                ← Authentication
    models.py         ← User, Patient, Doctor
    views.py          ← Login, register, profiles
    serializers.py    ← Validation
    permissions.py    ← Authorization classes
    urls.py           ← Auth routes

 appointments/         ← Booking system
    models.py         ← Appointment model
    views.py          ← REST endpoints
    serializers.py    ← Validation
    urls.py           ← Routes

 medical/              ← Medical records
 medications/          ← Medication tracking
 documents/            ← File uploads
 lab_tests/            ← Lab testing
 notifications/        ← In-app alerts

 requirements.txt      ← Python dependencies
 manage.py             ← Django CLI
```

Decision Summary Table

Decision	Choice	Key Rationale
Framework	Django + DRF	ORM + admin + security
Auth	Token-based	Decoupled SPA + API
Access Control	Role-based	Simpler than ABAC
Database	PostgreSQL	ACID + JSON + production-ready
User Models	Separate	Normalization + clarity
Prescriptions	Immutable	Legal + audit trail
Notifications	Dedicated app	Consistency + reusability
Appointment Prevention	DB constraint	Race condition safety
Pagination	20 items	UX + performance balance

Table 10.1: Architecture Decisions Summary

Appendix: Interview Preparation

Common Viva Questions & Answers

Q1: "Walk me through appointment booking flow"

Answer Pattern:

1. Frontend fetches available times
2. User selects time
3. Backend validates in order: auth, date, doctor availability, slot, double-book
4. Database unique constraint is last defense
5. Notifications automatically created
6. Both parties notified

Q2: "How do you prevent double-booking?"

Answer Pattern:

1. Serializer validates before creation
2. Database UNIQUE constraint on (doctor, date, time)
3. This prevents race conditions with concurrent requests
4. SQLAlchemy query cannot bypass DB constraint

Q3: "Why are prescriptions immutable?"

Answer Pattern:

1. Legal requirement: Proof of what was prescribed
2. Pharmacy already filled original prescription
3. Audit trail: Historical record for compliance
4. To correct, create new prescription with notes linking to original

Q4: "How does authentication work?"

Answer Pattern:

1. User logs in with email/password
2. Server validates credentials, generates token
3. Token stored in database, sent to frontend
4. Frontend includes token in Authorization header on every request
5. Server looks up token to identify user
6. If token invalid or missing → 401 Unauthorized

Q5: "What happens if I want to add a new role?"

Answer Pattern:

1. Add choice to User.user_type CharField
2. Create migration
3. Create new permission class (e.g., IsPharmacist)
4. Apply permission to relevant endpoints
5. Test with user of that role

Code Examples You Should Know

Permission Classes

```
@permission_classes([IsAuthenticated, IsDoctor])
def doctor_endpoint(request):
    # First checks: Is user logged in?
    # Second checks: Is user a doctor?
    pass
```

Creating Notifications

```
from notifications.helpers import create_notification

create_notification(
    recipient=doctor.user,
    title="New appointment",
    message="Patient X booked with you",
    notification_type="appointment",
    obj=appointment
)
```

Database Query Pattern

```
appointments = Appointment.objects.filter(
    doctor=user.doctor_profile,
    status='completed'
).select_related('patient', 'patient__user')
# select_related prevents N+1 queries
```

Validation Serializer

```
def validate(self, data):
    # Cross-field validation
    if invalid_condition:
        raise serializers.ValidationError({
            'field_name': "error message"
        })
    return data
```